Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г.В. Плеханова»

Отделение информационных технологий и программирования

Практическая работа №1

«Модульное тестирование в Visual Studio (MSTest)»

Работу выполнил: Зеленин Арсений

Группа: ИПо-21

Преподаватель: Берестов Дмитрий Борисович

Пермь

2026

Оглавление

1. Цель работы 3
2. Описание проекта 3
3. Структура решения 3
4. Целевая платформа 3
5. Ход выполнения работы 4
   5.1. Написание первого теста 4
   5.2. Исправление кода 4
   5.3. Добавление тестов для граничных условий 4
   5.4. Рефакторинг кода и тестов 5
   5.5. Дополнительные тесты 5
   5.6. Финальный запуск тестов 6
6. Полученные результаты 6
7. Выводы 6
8. Ссылки 7

Отчёт по практической работе «Модульное тестирование в Visual Studio (MSTest)»

1. Цель работы

Освоить методику создания и выполнения модульных тестов для управляемого кода на платформе .NET с использованием MSTest в среде Visual Studio. Научиться выявлять и исправлять ошибки с помощью тестов, а также применять подход разработки на основе тестирования (TDD).

2. Описание проекта

Был разработан проект Bank, содержащий класс BankAccount, моделирующий банковский счёт. Класс предоставляет методы для списания средств (Debit), пополнения (Credit), проверки возможности списания (CanDebit) и получения баланса (Balance). Исходно в методе Debit была допущена логическая ошибка: вместо вычитания суммы выполнялось сложение. Для проверки корректности работы создан тестовый проект BankTests с использованием фреймворка MSTest.

3. Структура решения

```
Bank-UnitTesting-Zelenin/
├── Bank.slnx
├── Bank/
│   ├── Bank.csproj
│   └── BankAccount.cs
├── BankTests/
│   ├── BankTests.csproj
│   ├── MSTestSettings.cs
│   └── BankAccountTests.cs
├── README.md
└── Report.md
```

4. Целевая платформа

Оба проекта ориентированы на .NET 10.0. Используется мета-пакет MSTest версии 4.0.1. В файле MSTestSettings.cs включён параллельный запуск тестов на уровне методов:

```
[assembly: Parallelize(Scope = ExecutionScope.MethodLevel)]
```

5. Ход выполнения работы

5.1. Написание первого теста

Был создан метод Debit_WithValidAmount_UpdatesBalance, который проверяет, что после списания допустимой суммы баланс уменьшается на заданную величину. При первом запуске тест завершился ошибкой, так как в коде использовалось сложение вместо вычитания.

```
[TestMethod]
public void Debit_WithValidAmount_UpdatesBalance()
{
    var account = new BankAccount("Arseniy Zelenin", 20.00);

    account.Debit(6.35);

    Assert.AreEqual(13.65, account.Balance, 0.001, "Account not debited correctly");
}
```

5.2. Исправление кода

В методе Debit класса BankAccount строка balance += amount; была заменена на balance -= amount;. После этого тест стал проходить успешно.

5.3. Добавление тестов для граничных условий

Были добавлены два теста, проверяющие выброс исключений при некорректных аргументах: при отрицательной сумме и при сумме, превышающей баланс. Полные имена методов приведены в листинге ниже.

В MSTest 4 для проверки исключений используется метод Assert.ThrowsExactly, который возвращает перехваченное исключение и позволяет дополнительно проверить его сообщение:

```
[TestMethod]
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
{
    var account = new BankAccount("Arseniy Zelenin", 20.00);

    var exception = Assert.ThrowsExactly<ArgumentOutOfRangeException>(() =>
    account.Debit(-50.00));

    StringAssert.Contains(exception.Message,
    BankAccount.DebitAmountLessThanZeroMessage);
}
```

5.4. Рефакторинг кода и тестов

В класс BankAccount были добавлены публичные константы для сообщений об ошибках:

```
public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
public const string DebitAmountLessThanZeroMessage = "Debit amount is less than zero";
public const string CreditAmountLessThanZeroMessage = "Credit amount is less than
    zero";
```

Конструктор исключения в методе Debit был заменён на версию с сообщением:

```
throw new ArgumentOutOfRangeException(nameof(amount), amount,
    DebitAmountLessThanZeroMessage);
```

Тесты были дополнены проверкой корректности сообщения, что повысило надёжность проверок: теперь тест падает не только при отсутствии исключения, но и при неверном тексте ошибки.

5.5. Дополнительные тесты

Для расширения покрытия добавлены тесты пополнения счёта и параметризованный тест метода CanDebit с атрибутом [DataRow]:

```
[TestMethod]
public void Credit_WithValidAmount_IncreasesBalance()
{
    var account = new BankAccount("Arseniy Zelenin", 20.00);

    account.Credit(8.25);

    Assert.AreEqual(28.25, account.Balance, 0.001, "Account not credited correctly");
}

[TestMethod]
[DataRow(5.00, true)]
[DataRow(20.00, true)]
[DataRow(21.00, false)]
[DataRow(-1.00, false)]
public void CanDebit_ReturnsExpectedAvailability(double amount, bool expected)
{
    var account = new BankAccount("Arseniy Zelenin", 20.00);

    Assert.AreEqual(expected, account.CanDebit(amount));
}
```

Параметризованный тест проверяет граничные значения: сумму меньше баланса, сумму, равную балансу, сумму больше баланса и отрицательную сумму.

5.6. Финальный запуск тестов

После всех исправлений тесты запускаются командой dotnet test. В наборе шесть тестовых методов, один из которых параметризован четырьмя наборами данных, поэтому обозреватель тестов показывает девять успешно пройденных проверок:

```
dotnet test Bank.slnx -c Release

Пройден! : не пройдено 0, пройдено 9, пропущено 0, всего 9,
           длительность 61 ms - ZeleninBankTests.dll (net10.0)
```

6. Полученные результаты

Разработан класс BankAccount с корректной логикой списания и пополнения средств.

Создан набор модульных тестов, покрывающих следующие сценарии:

- успешное списание допустимой суммы;
- попытку списания отрицательной суммы;
- попытку списания суммы, превышающей баланс;
- успешное пополнение счёта;
- попытку пополнения отрицательной суммы;
- проверку метода CanDebit для граничных значений (четыре набора данных).

Все тесты выполняются успешно, что подтверждает правильность реализации класса BankAccount.

7. Выводы

В ходе работы были освоены:

- создание тестовых проектов в Visual Studio;
- написание и запуск модульных тестов с помощью MSTest;
- использование утверждений Assert.AreEqual, Assert.ThrowsExactly и StringAssert.Contains;
- параметризация тестов с помощью атрибута [DataRow];
- обработка и проверка исключений в тестах;

- подход «красный — зелёный — рефакторинг» (TDD).

Модульное тестирование позволило своевременно выявить и исправить ошибку, а также обеспечить защиту от регрессий при дальнейших изменениях кода. Отдельным результатом стало понимание того, что информативные сообщения об ошибках и публичные константы делают тесты устойчивыми к изменению текста исключений.

8. Ссылки

Документация MSTest: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest